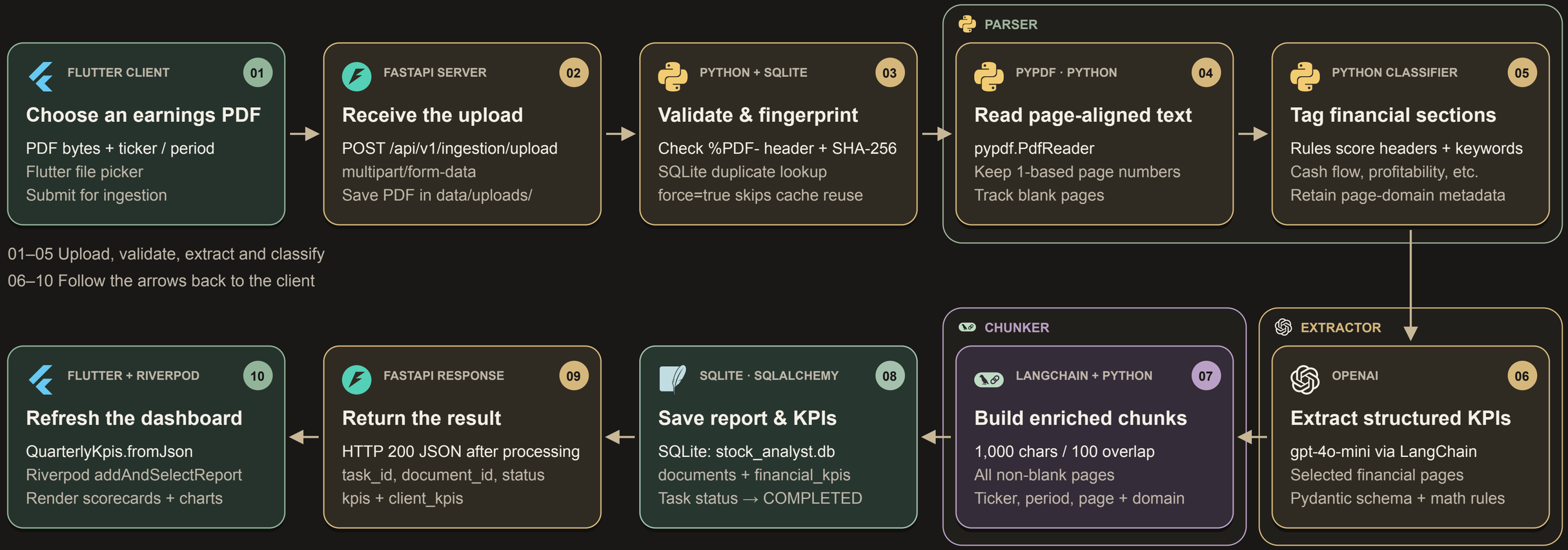
From earnings PDF to usable evidence

The implemented ingestion flow, from Flutter upload to server processing and back to the dashboard.

CLIENT · SERVER PROCESSING · RAG PREPARATION · PERSISTED OUTPUTS



01–05 Upload, validate, extract and classify

06–10 Follow the arrows back to the client

INSPECTABLE OUTPUTS · server/.artifact/

Pages → section tags → raw splits → enriched chunks → KPI extraction → persistence snapshot → run summary

RAG boundary: chunk preparation is implemented. Embeddings, vector indexing, retrieval and answer generation are next.

What makes ingestion useful for RAG?

Turn a PDF into small, traceable pieces of evidence before adding embeddings and retrieval.

PARSER

PYPDF · PYTHON — 01

Keep page boundaries

pypdf extracts page text
Page numbers start at 1
Blank pages are skipped later

PYTHON CLASSIFIER — 02

Keep financial context

Rule-based domain labels
Headers + body keywords
Each page keeps its domain

EXTRACTOR

OPENAI · GPT-4O-MINI — 03

Resolve report identity

KPIs supply canonical metadata
Ticker + fiscal quarter + year
Statement pages sent to LLM

CHUNKER

LANGCHAIN — 04

Split within each page

RecursiveCharacterTextSplitter
1,000 characters per chunk
100-character overlap

PYTHON ENRICHMENT — 05

Attach source metadata

Stable chunk ID + text
Ticker, period, page + domain
Chunk position + char count

Chunking uses all non-blank pages, including narrative pages outside the KPI extraction subset.

A SAVED CHUNK · actual inspection-artifact metadata

```
chunk_id: "TSLA_Q2_2026_p3_c0"
ticker / period: "TSLA" / "Q2" / 2026
page_number: 3
financial_domain: "cash_flow"
chunk position: 0 of 4 · char_count: 943
```

WHERE THE EVIDENCE LIVES TODAY

EnrichedChunk objects in memory

03b_enriched_chunks_*.json / .md

Written when save_artifacts=true

The Flutter upload enables artifact saving.

NEXT RAG STAGE · planned, outside the current ingestion implementation

Enriched chunks → embeddings → vector index → retrieve relevant evidence → generate a cited answer